

MINOR ASSIGNMENT- 3: NATURAL LANGUAGE PROCESSING

1. Define Natural Language Processing (NLP). Provide three real-world applications of NLP and explain how they impact society.

Ans - **Natural Language Processing (NLP)** is a branch of artificial intelligence (AI) that enables computers to understand, interpret, and generate human language. It combines linguistics, machine learning, and computational techniques to process text and speech in a way that mimics human communication.

Real-World Applications of NLP

1. Machine Translation (e.g., Google Translate)

Impact: Breaks language barriers by enabling real-time translation of text and speech between different languages. This enhances global communication, education, and business collaborations.

2. Virtual Assistants (e.g. Siri , Alexa , Google Assistant)

Impact: Improves accessibility by allowing users to interact with technology using voice commands. Helps in daily task like setting reminders, searching for information, and controlling smart devices.

3. Sentiment Analysis(e.g. Social Media Monitoring)

Impact: Business analyze customer feedback, social media posts, and product reviews to understand public opinion. Helps companies improve services and detect emerging trends or crises.

2. Explain the following terms and their significance in NLP:

- Tokenization
- Stemming
- Lemmatization

Ans.-

Tokenization: is the process of breaking text into smaller units called tokens which can be words, phrases or sentences.

Significance:

- helps in text preprocessing by converting raw text into meaningful chunks.
- Eg: Input : "Natural Language Processing is amazing!"

Word Tokenization: ["Natural", "Language", "Processing", "is", "amazing", "!",]

Stemming: reduces words to their root form by stripping suffixes using heuristic rules

Significance:

- speeds up processing by reducing word variations
- useful in search engines to match different word forms(eg running -> run)
- less accurate as it may produce no-existent word (eg - runn , runnin, runni)

Lemmatization: reduces word to their base or dictionary form (lemma) using linguistic rules.

Significance:

-more accurate than stemming as it considers context and word meaning.

-useful in NLP applications like chatbots and machine translation.

-eg Input: running, better, mice

Output: run , good , mouse

Q3. What is Part-of-Speech (POS) tagging? Discuss its importance with an example

Ans-Part-of-Speech (POS) Tagging is a fundamental step in Natural Language Processing (NLP) where each word in a sentence is labeled with its grammatical role, such as noun, verb, adjective, adverb, etc. The process helps machines understand the syntactic structure of sentences

Importance of POS Tagging:

a) **Text Analysis and Understanding:** POS tagging enables deeper linguistic analysis, helping machines grasp context and meaning in text.

b) **Named Entity Recognition (NER):** Identifying proper nouns (e.g., names, places) becomes easier with accurate POS tagging.

c) **Syntactic Parsing:** Helps in identifying grammatical relationships and sentence structure.

d) **Machine Translation and Chat-bots:** Ensures accurate translation and response generation by recognizing the role of each word.

Example: Consider the sentence: "She enjoys a long walk in the morning."

After POS tagging, it would look like:

She (PRON)- Pronoun enjoys (VERB)- Verb a (DET)- Determiner long (ADJ)- Adjective walk (NOUN)- Noun in (PREP)- Preposition the (DET)- Determiner morning (NOUN)- Noun

Q4. Create a TextBlob named exercise_blob containing "This is a TextBlob"

```
#Name:Arman Singh
```

```
#Regd-2241019468
```

```
from textblob import TextBlob
```

```
exercise_blob = TextBlob("This is a TextBlob")
```

```
print(exercise_blob)
```

Output:

```
This is a TextBlob
```

5. Write a Python script to perform the following tasks on the given text:

- Tokenize the text into words and sentences.
- Perform stemming and lemmatization using NLTK or SpaCy.
- Remove stop words from the text.
- Sample Text:
"Natural Language Processing enables machines to understand and process human languages. It is a fascinating field with numerous applications, such as chatbots and language translation."

```
#Name-Arman Singh
#Regd-2241019468
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import stopwords

# Download necessary NLTK resources
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('stopwords')

# Sample text
text = """Natural Language Processing enables machines to understand and process
It is a fascinating field with numerous applications, such as chatbots and language translation"""

# Tokenization
words = word_tokenize(text)
sentences = sent_tokenize(text)

# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]

# Removing Stop Words
stop_words = set(stopwords.words("english"))
filtered_words = [word for word in words if word.lower() not in stop_words]

# Output results
print("Tokenized Words:", words)
print("Tokenized Sentences:", sentences)
print("Stemmed Words:", stemmed_words)
print("Lemmatized Words:", lemmatized_words)
print("Filtered Words (without Stop Words):", filtered_words)
```

text = " "
blob = TextBlob(text)
blob.sentences
blob.words

for token in word_tokens:
w = Word(token)
print(f'{token:15} --> stem:
{w.stem():10} | lemma:
{w.lemmatize()}')

Output

```
Tokenized Words: ['Natural', 'Language', 'Processing', 'enables', 'machines', 'to', 'u  
nderstand', 'and', 'process', 'human', 'languages', '.', 'It', 'is', 'a', 'fascinating  
, 'field', 'with', 'numerous', 'applications', ',', 'such', 'as', 'chatbots', 'and',  
'language', 'translation', '.']
Tokenized Sentences: ['Natural Language Processing enables machines to understand and  
process human languages.', 'It is a fascinating field with numerous applications, such  
as chatbots and language translation.']
Stemmed Words: ['natur', 'languag', 'process', 'enabl', 'machin', 'to', 'understand',  
'and', 'process', 'human', 'languag', '.', 'it', 'is', 'a', 'fascin', 'field', 'with',  
'numer', 'applic', ',', 'such', 'as', 'chatbot', 'and', 'languag', 'translat', '.']
Lemmatized Words: ['Natural', 'Language', 'Processing', 'enables', 'machine', 'to', 'u  
nderstand', 'and', 'process', 'human', 'language', '.', 'It', 'is', 'a', 'fascinating'  
, 'field', 'with', 'numerous', 'application', ',', 'such', 'a', 'chatbots', 'and', 'la  
nguage', 'translation', '.']
Filtered Words (without Stop Words): ['Natural', 'Language', 'Processing', 'enables',  
'machines', 'understand', 'process', 'human', 'languages', '.', 'fascinating', 'field'  
, 'numerous', 'applications', ',', 'chatbots', 'language', 'translation', '.']
```

6. Web Scraping with the Requests and BeautifulSoup Libraries:

- Use the **requests** library to download the *www.python.org* home page's content.
- Use the **Beautiful Soup** library to extract only the text from the page.
- Eliminate the stop words in the resulting text, then use the **wordcloud** module to create a word cloud based on the text.

Ans-

```
#Name - Arman Singh
#Regd- 2241019468
import requests
from bs4 import BeautifulSoup
from wordcloud import WordCloud
import nltk
from nltk.corpus import stopwords

# Download stopwords
nltk.download("stopwords")
stop_words = set(stopwords.words("english"))

# Step 1: Download www.python.org content
url = "https://www.python.org/"
response = requests.get(url)

# Step 2: Extract text using BeautifulSoup
soup = BeautifulSoup(response.text, "html.parser")
text = soup.get_text()

# Step 3: Remove stop words
words = text.split()
filtered_text = " ".join(word for word in words if word.lower() not in stop_words)

# Step 4: Generate a Word Cloud
wordcloud = WordCloud(width=800, height=400, background_color="white").generate(
    filtered_text

# Step 5: Display the Word Cloud
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.show()
```

Output:



7. **(Tokenizing Text and Noun Phrases)** Using the text from above problem, create a TextBlob, then tokenize it into Sentences and Words, and extract its noun phrases.

Ans-

```
#Name-Arman Singh
#Regd-2241019468
from textblob import TextBlob
import requests
from bs4 import BeautifulSoup
import nltk
nltk.download('punkt')

# Problem 7: Tokenizing Text and Extracting Noun Phrases
text = "Natural Language Processing enables machines to understand and process human languages."
blob = TextBlob(text)

print("\nSentences:", blob.sentences)
print("\nWords:", blob.words)
print("\nNoun Phrases:", blob.noun_phrases)
```

Output

```
Sentences: [Sentence("Natural Language Processing enables machines to understand and process human languages.")]

Words: ['Natural', 'Language', 'Processing', 'enables', 'machines', 'to', 'understand', 'and', 'process', 'human', 'languages']

Noun Phrases: ['language processing', 'process human languages']
```

8. **(Sentiment of a News Article)** Using the techniques in problem no. 5, download a web page for a current news article and create a TextBlob. Display the sentiment for the entire TextBlob and for each Sentence.

Ans-

```
#Name-Arman Singh
#Regd-2241019468
from textblob import TextBlob
import requests
from bs4 import BeautifulSoup
import nltk
nltk.download('punkt')
url = "https://www.bbc.com/news" # Example news website
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

news_text = soup.get_text()
news_blob = TextBlob(news_text)

print("\nOverall Sentiment of the News Article:", news_blob.sentiment)

# Sentiment for each sentence
for sentence in news_blob.sentences[:5]: # Limiting to first 5 sentences
    print(f"Sentence: {sentence}\nSentiment: {sentence.sentiment}\n")
```

Output:


```
Overall Sentiment of the News Article: Sentiment(polarity=0.1192929292929293, subjectivity=0.41234006734006723)
Sentence: Home - BBC NewsSkip to contentBritish Broadcasting CorporationHomeNewsSportBusinessInnovationCultureArtsTravelEarthAudioVideoLiveIsrael-Gaza WarWar in UkraineUS & CanadaUKAfricaAsiaAustraliaEuropeLatin AmericaMiddle EastIn PicturesBBC InDepthBBC VerifyHomeNewsIsrael-Gaza WarWar in UkraineUS & CanadaUKUK PoliticsEnglandN.
Sentiment: Sentiment(polarity=0.0, subjectivity=0.0)

Sentence: IrelandN.
Sentiment: Sentiment(polarity=0.0, subjectivity=0.0)

Sentence: Ireland PoliticsScotlandScotland PoliticsWalesWales PoliticsAfricaAsiaChinaIndiaAustraliaEuropeLatin AmericaMiddle EastIn PicturesBBC InDepthBBC VerifySportBusiness
```

9. (Sentiment of a News Article with the NaiveBayesAnalyzer) Repeat the previous exercise but use the NaiveBayesAnalyzer for sentiment analysis.

Ans-

```
#Name-Arman Singh
#Regd-2241019468
import requests
from bs4 import BeautifulSoup
from textblob import TextBlob
from textblob import TextBlob
from textblob.sentiments import NaiveBayesAnalyzer
url = "https://www.bbc.com/news" # You can change this to any news URL
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

text = soup.get_text()
blob = TextBlob(text, analyzer=NaiveBayesAnalyzer())

print("Overall Sentiment:", blob.sentiment)

for sentence in blob.sentences[:5]: # Analyzing only the first 5 sentences
    print("\nSentence:", sentence)
    print("Sentiment:", sentence.sentiment)
```

Output

```
Sentiment: Sentiment(classification='pos', p_pos=0.9999972048244744, p_neg=2.7951755274497726e-06)

Sentence: 13 hrs agoUS & CanadaHow will India navigate a world on the brink of a trade war?India is the world's fastest-growing major economy.
Sentiment: Sentiment(classification='pos', p_pos=0.9940625934029768, p_neg=0.005937406597024563)

Sentence: How it deals with Trump will define its economic trajectory.6 hrs agoAsia'I could live 30 years - but want to die': Has assisted dying in Canada gone too far?BBC News reports on assisted dying in Canada, where some say it's now easier to choose to die than get support to live2 hrs agoUS & CanadaInfluencers 'new' threat to uncontacted tribes, warns group after US tourist arrest8 hrs agoAsiaHezbollah at crossroads after blows from war weaken group3 hrs agoMiddle EastHungary withdraws from International Criminal Court during Netanyahu visit10 hrs agoWorldWho might buy TikTok as ban deadline looms?
Sentiment: Sentiment(classification='pos', p_pos=0.9980220813974566, p_neg=0.0019779186025442115)
```

10. **(Spell Check a Project Gutenberg Book)** Download a Project Gutenberg book and create a TextBlob. Tokenize the TextBlob into Words and determine whether any are misspelled. If so, display the possible corrections.

Ans-

```
#Name-Arman Singh
#Regd-2241019468
from textblob import TextBlob
import requests

url = "https://www.gutenberg.org/files/1342/1342-0.txt" # Example: "Pride and P
response = requests.get(url)
text = response.text

blob = TextBlob(text)

misspelled_words = [word for word in blob.words[:100] if word.correct() != word]

print("Misspelled Words and Corrections:")
for word in misspelled_words:
    print(f"{word} -> {TextBlob(word).correct()}")
```



Output:

```
----- REGD-2241019468
Misspelled Words and Corrections:
OF -> of
Reading -> Leading
Jane -> Lane
Jane -> Lane
Austen -> Listen
Ruskin -> Skin
Charing -> Sharing
Allen -> Ellen
CO -> of
Comyns -> Romans
Carr -> Parr
Walt -> Salt
|
```

- 11.
- Write a Python program that takes user input in English and translates it to French, Spanish, and German using TextBlob.
 - Create a program that takes multiple user-inputted sentences, analyzes polarity and subjectivity, and categorizes them as objective/subjective and positive/negative/neutral.
 - Develop a function that takes a paragraph, splits it into sentences, and calculates the sentiment score for each sentence individually.
 - Write a program that takes a sentence as input and prints each word along with its POS tag using TextBlob.
 - Create a function that takes a user-inputted word, checks its spelling using TextBlob, and suggests top 3 closest words if a mistake is found.
 - Build a Python script that extracts all adjectives from a given paragraph and prints them in order of occurrence.
 - Write a program that takes a news article as input and extracts the top 5 most common noun phrases as keywords.
 - Write a program that takes a news article as input and extracts the top 5 most common noun phrases as keywords.
 - Write a program that summarizes a given paragraph by keeping only the most informative sentences, based on noun phrase frequency.

Ans-

```
#Name-Arman Singh
#Regd-2241019468
from textblob import TextBlob
from collections import Counter
from deep_translator import GoogleTranslator
def translate_text(text):
    print("French:", GoogleTranslator(source='auto', target='fr').translate(text))
    print("Spanish:", GoogleTranslator(source='auto', target='es').translate(text))
    print("German:", GoogleTranslator(source='auto', target='de').translate(text))
def analyze_sentiment(text):
    blob = TextBlob(text)
    for sentence in blob.sentences:
        print(f"{sentence} -> Polarity: {sentence.sentiment.polarity}, Subjectivity: {sentence.sentiment.subjectivity}")
def sentence_sentiment(paragraph):
    blob = TextBlob(paragraph)
    for sentence in blob.sentences:
        print(f"{sentence} -> Sentiment Score: {sentence.sentiment}")
def pos_tagging(text):
    blob = TextBlob(text)
    for word, tag in blob.tags:
        print(f"{word} -> {tag}")
def spell_check(word):
    blob = TextBlob(word)
    suggestions = blob.correct()
    print("Suggested Correction:", suggestions)
def extract_adjectives(paragraph):
    blob = TextBlob(paragraph)
    adjectives = [word for word, tag in blob.tags if tag == "JJ"]
    print("Adjectives:", adjectives)
def extract_keywords(text):
    blob = TextBlob(text)
    noun_phrases = blob.noun_phrases
    common_phrases = Counter(noun_phrases).most_common(5)
    print("Top 5 Keywords:", common_phrases)
def summarize_text(paragraph):
    blob = TextBlob(paragraph)
    scores = {sentence: sum(blob.noun_phrases.count(phrase) for phrase in sentence) for sentence in blob.sentences}
    summary = sorted(scores, key=scores.get, reverse=True)[:3]
    print("Summary:", ' '.join(str(s) for s in summary))
text = input("Enter a paragraph: ")
translate_text(text)
analyze_sentiment(text)
sentence_sentiment(text)
pos_tagging(text)
word = input("Enter a word to check spelling: ")
spell_check(word)
extract_adjectives(text)
extract_keywords(text)
summarize_text(text)
```

Output

```
Enter a paragraph: Arman is best
French: Arman est le meilleur
Spanish: Arman es mejor
German: Arman ist am besten
Arman is best -> Polarity: 1.0, Subjectivity: 0.3
Arman is best -> Sentiment Score: Sentiment(polarity=1.0, subjectivity=0.3)
Arman -> NNP
is -> VBZ
best -> JJS
Enter a word to check spelling: Crossiant
Suggested Correction: Crossiant
Adjectives: []
Top 5 Keywords: [('arman', 1)]
Summary: Arman is best
```


12. Write a Python program that takes a word as input and returns:

- Its definition
- Its synonyms
- Its antonyms(if available)

Ans-

```
#Name - Arman Singh
#Regd - 2241019468
from nltk.corpus import wordnet

def word_analysis(word):
    synonyms = []
    antonyms = []
    definitions = []

    for syn in wordnet.synsets(word):
        definitions.append(syn.definition())
        for lemma in syn.lemmas():
            synonyms.append(lemma.name())
            if lemma.antonyms():
                antonyms.append(lemma.antonyms()[0].name())

    print("Definitions:", definitions)
    print("Synonyms:", set(synonyms))
    print("Antonyms:", set(antonyms))

word = input("Enter a word: ")
word_analysis(word)
```

Output:

```
Enter a word: Perspective
Definitions: ['a way of regarding situations or topics etc.', 'the appearance of things relative to one another as determined by their distance from the viewer']
Synonyms: {'position', 'perspective', 'view', 'linear_perspective'}
Antonyms: set()
|
```

13. • Write a Python program that reads a .txt file, processes the text, and generates a word cloud visualization.
- Create a word cloud in the shape of an object (e.g., a heart, star) using WordCloud and a mask image.

```
#Name: Arman Singh
#RegdNo-2241019468
from wordcloud import WordCloud
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

file_path = r"D:/Books 3rd Year 6th Sem/Python/MyAssignment/Ass3/file.txt"
mask_path = r"D:/Books 3rd Year 6th Sem/Python/MyAssignment/Ass3/heart.png"

with open(file_path, "r", encoding="utf-8") as file:
    text = file.read()

mask = np.array(Image.open(mask_path))

wordcloud = WordCloud(width=800, height=400, background_color="white", mask=mask)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.show()
```

Output:



14. (Textatistic: Readability of News Articles) Using the above techniques, download from several news sites current news articles on the same topic. Perform readability assessments on them to determine which sites are the most readable. For each article, calculate the average number of words per sentence, the average number of characters per word and the average number of syllables per word.

#Name: Arman Singh
#RegdNo-2241019468

```
import requests
from bs4 import BeautifulSoup
import textstat
urls = [
    "https://www.bbc.com/news/world",
    "https://edition.cnn.com/world",
    "https://www.aljazeera.com/news/"
]

def analyze_readability(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'html.parser')
    text = ' '.join([p.get_text() for p in soup.find_all('p')])
    readability = {
        'avg_words_per_sentence': textstat.avg_sentence_length(text),
        'avg_chars_per_word': textstat.avg_character_per_word(text),
        'avg_syllables_per_word': textstat.avg_syllables_per_word(text)
    }
    print("URL:", url)
    print("Average Words per Sentence:", readability['avg_words_per_sentence'])
    print("Average Characters per Word:", readability['avg_chars_per_word'])
    print("Average Syllables per Word:", readability['avg_syllables_per_word'])
    for url in urls:
        analyze_readability(url)
```

Output:

```
URL: https://www.bbc.com/news/world
Average Words per Sentence: 15.3
Average Characters per Word: 5.16
Average Syllables per Word: 1.6
URL: https://edition.cnn.com/world
Average Words per Sentence: 10.0
Average Characters per Word: 4.43
Average Syllables per Word: 1.3
URL: https://www.aljazeera.com/news/
Average Words per Sentence: 16.9
Average Characters per Word: 5.22
Average Syllables per Word: 1.7
```

15. **(spaCy: Named Entity Recognition)** Using the above techniques, download a current news article, then use the spaCy library's named entity recognition capabilities to display the named entities (people, places, organizations, etc.) in the article.

```
# Name - Arman Singh
# Regd. no - 2241019468
import spacy

nlp = spacy.load("en_core_web_sm")
text = "Apple Inc. is a tech company founded by Steve Jobs in California."
doc = nlp(text)

for ent in doc.ents:
    print(f"{ent.text}: {ent.label_}")
```

Output:

```
Apple Inc.: ORG
Steve Jobs: PERSON
California: GPE
|
```

16. **(spaCy: Shakespeare Similarity Detection)** Using the spaCy techniques, download a Shakespeare comedy from Project Gutenberg and compare it for similarity with Romeo and Juliet.

```
#Name - Arman Singh
#Regd - 2241019468
import spacy
from urllib.request import urlopen

nlp = spacy.load("en_core_web_md")

def fetch_text(url):
    response = urlopen(url)
    return response.read().decode("utf-8")

romeo_juliet_url = "https://www.gutenberg.org/cache/epub/1513/pg1513.txt"
comedy_url = "https://www.gutenberg.org/cache/epub/1524/pg1524.txt"

romeo_juliet_text = nlp(fetch_text(romeo_juliet_url))
comedy_text = nlp(fetch_text(comedy_url))

similarity_score = romeo_juliet_text.similarity(comedy_text)

print(f"Similarity between Romeo and Juliet and the comedy: {similarity_score:.2f}")
```

Output:

```
Similarity between Romeo and Juliet and the comedy: 1.00
>|
```

17. (**textblob.utils Utility Functions**) Use **strip_punc** and **lowerstrip** functions of TextBlob's **textblob.utils** module with **all=True** keyword argument to remove punctuation and to get a string in all lowercase letters with whitespace and punctuation removed. Experiment with each function on Romeo and Juliet.

```
#Name - Arman Singh
#Regd - 2241019468
from textblob.utils import strip_punc, lowerstrip
from urllib.request import urlopen

url = "https://www.gutenberg.org/cache/epub/1513/pg1513.txt"
response = urlopen(url)
text = response.read().decode("utf-8")

cleaned_text_strip = strip_punc(text, all=True)
cleaned_text_lowerstrip = lowerstrip(text, all=True)

print("After strip_punc:\n", cleaned_text_strip[:500])
print("\nAfter lowerstrip:\n", cleaned_text_lowerstrip[:500])
```

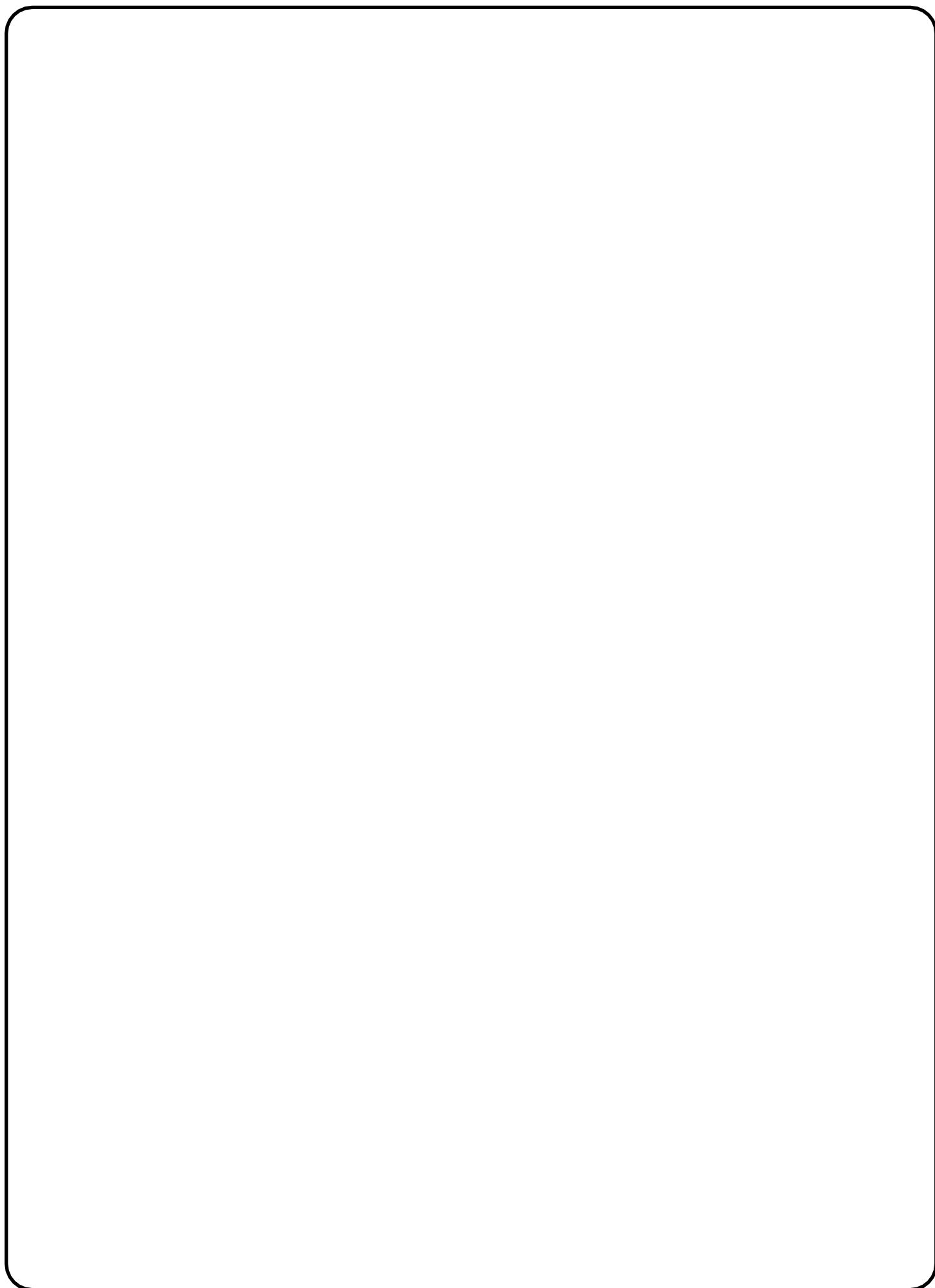
Output:

```
After strip_punc:
The Project Gutenberg eBook of Romeo and Juliet
```

```
This ebook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever You may copy it give it away or reuse it under the terms
of the Project Gutenberg License included with this ebook or online
at wwwgutenbergorg If you are not located in the United States
you will have to check the laws of the country where you are located
before using this eBook
```

```
After lowerstrip:
the project gutenberg ebook of romeo and juliet
```

```
this ebook is for the use of anyone anywhere in the united states and
most other parts of the world at no cost and with almost no restrictions
whatsoever you may copy it give it away or reuse it under the terms
of the project gutenberg license included with this ebook or online
at wwwgutenbergorg if you are not located in the united states
you will have to check the laws of the country where you are located
before using this ebook
```

A large, empty rectangular box with rounded corners, likely a placeholder for a drawing or diagram. The box is defined by a thin black border and occupies the majority of the page area below the header.